ADXL362 accelerometer interface

Logic Design Using HDL | Laboratory 10 | Register-level acquisition and UART telemetry

Design objective

Design and verify a synchronous Verilog controller for the ADXL362 integrated on the Nexys4 DDR. The controller shall configure the sensor over SPI, acquire three signed acceleration components in a single register burst, and transmit one ASCII record per acquisition to a host computer.

Design parameter	Specification
Acquisition	12-bit signed XYZ, ±2 g range, 100 Hz sensor output data rate
FPGA interface	100 MHz system clock, four-wire SPI mode 0, 1 MHz serial clock
Host interface	10 records/s, fixed-width signed decimal, UART at 115200 baud, 8N1

Verification objective

Relate each field in a received record to its sensor register and SPI bit position. Verify the acquisition interval, signed limits and serial framing against the RTL and the measured output.

Measurement model and sensitivity

The ADXL362 resolves specific force along three orthogonal axes fixed to the sensor package.

Capacitive MEMS transduction

Acceleration displaces a suspended proof mass relative to the substrate. Differential capacitance changes with displacement. The sensor electronics convert and filter the signal and periodically update one signed digital result per axis.

Static response

A sensor supported on a bench measures the projection of the support acceleration onto its axes. For an ideal stationary device the vector magnitude is 1 g. If an axis makes an angle theta with the upward vertical, its ideal component is cos(theta) in g.

```
n = S a + b + e        S ~ 1000 counts/g at ±2 g
```

Here n is the axis code, a is the signed component in g, b is the zero-g offset in counts, and e includes noise and quantization. Reversing the axis changes the sign of a while retaining the offset.

Sensor register organization

The ADXL362 exposes a byte-addressed register map through its SPI port.

Each address selects an eight-bit location inside the sensor. The data sheet assigns a symbolic name and access permissions to that location. For example, address 0x0E selects XDATA_L, the read-only register containing bits [7:0] of the most recent X result.

Register class	Address example	Source of contents	FPGA operation
Output data	0x0E / XDATA_L	Sensor conversion logic	Read latest value
Configuration	0x2C / FILTER_CTL	Reset default or FPGA write	Set range and update rate
Command	0x1F / SOFT_RESET	Defined write code	Write 0x52 to initiate reset

Address, contents and local storage

Two successive conversions may place 0x58 and 0x5C at the same address 0x0E. On a read, the FPGA supplies the address on MOSI and captures the contents on MISO. Its local x register changes only when the acquisition FSM commits the completed XYZ burst.

XYZ output register map

An axis has 12 significant bits. The sensor makes them available through two adjacent byte addresses.

Address	Register	Example byte	Meaning
0x0E	XDATA_L	0x58	X bits 7 through 0
0x0F	XDATA_H	0x00	X bits 11 through 8 in the low nibble
0x10	YDATA_L	0x83	Y bits 7 through 0
0x11	YDATA_H	0x00	Y bits 11 through 8 in the low nibble
0x12	ZDATA_L	0x67	Z bits 7 through 0
0x13	ZDATA_H	0xFC	Z bits 11 through 8 in the low nibble

L and H mean low and high. A nibble is four bits. In each high byte, the upper nibble repeats the sign bit. The table reconstructs a possible register image from the first recorded terminal line. The SPI wires were not recorded during that hardware run.

Reconstruction rule `signed_axis[11:0] = {high_byte[3:0], low_byte[7:0]}`

Initialization and measurement mode

The FPGA writes configuration registers before it reads any output registers.

Step	Register and address	Value	Effect
1	SOFT_RESET / 0x1F	0x52	Restores defaults and puts the chip in standby.
2	FILTER_CTL / 0x2C	0x03	Selects ±2 g and 100 measurement updates per second.
3	POWER_CTL / 0x2D	0x02	Starts continuous measurement.

Configuration fields and settling time

0x03 is 00000011 in binary. In FILTER_CTL, bits [7:6] = 00 select the ±2 g range and bits [2:0] = 011 select 100 Hz. In POWER_CTL, bits [1:0] = 10 select measurement mode. A value gets its meaning from the address at which the FPGA writes it.

The FPGA waits 10 ms at startup and 1 ms after soft reset. Once it enables measurement, the specified settling interval is 4/ODR, or 40 ms at 100 Hz. This design allows 100 ms before reading.

SPI transaction format and clock phase

SPI is a synchronous serial interface. The FPGA supplies the clock for both directions.

Signal	Direction	Electrical and protocol function
CS_N	FPGA to sensor	Low selects the chip and holds one transaction open.
SCLK	FPGA to sensor	Each rising edge samples one bit. Clock idles low (mode 0).
MOSI	FPGA to sensor	Carries command, register address and write data, MSB first.
MISO	Sensor to FPGA	Carries the requested bytes while the FPGA supplies clocks.

Example write / start measuring

```
MOSI      0A              2D              02
```

Read from left to right: 0A requests a write, 2D selects POWER_CTL and 02 is the value to store there. The FPGA lowers CS_N before bit 7 of 0A and raises it after all 24 bits.

Sequential addressing in an XYZ burst

Send the start address once. The ADXL362 advances to the next address after each data byte.

Byte slot	0	1	2	3	4	5	6	7
MOSI	0B	0E	00	00	00	00	00	00
MISO example	--	--	58	00	83	00	67	FC
Register read	--	--	0E	0F	10	11	12	13

During the six dummy bytes the controller generates 48 additional clock pulses. During those pulses, the sensor returns the six requested bytes on MISO. Keep CS_N low for all eight slots. Ignore the first two incoming bytes while the command and address travel to the sensor.

Edge-indexed verification

With the first rising edge numbered 1, edges 1-8 send the command and 9-16 send the address. Edge 17 captures bit 7 of XDATA_L. Edge 64 captures bit 0 of ZDATA_H. Verify both boundaries against CS_N in simulation.

Transaction sequencer and SPI bit engine

The top-level finite-state machine (FSM) chooses the operation. The SPI engine performs one transaction at a time.

DELAY: wait > **ISSUE: start frame** > **TRANSFER: wait for done**

DELAY expires into ISSUE. ISSUE pulses start for one system clock. TRANSFER returns to DELAY on done. After initialization, it schedules repeated reads. On completion of a read it latches all three axes on one FPGA clock edge and pulses report to start the text formatter.

IDLE: assert CS > **RISE: sample MISO** > **FALL: advance bit** > **RELEASE_CS: done**

In spi_frame.v, RISE and FALL repeat until the requested number of bits has shifted. FALL returns to RISE if bits remain. After the final falling edge the engine waits, releases CS and pulses done. All state registers use the 100 MHz board clock. SCLK is an output driven by the engine.

Clock division, throughput and sampling

Derive the bus capacity and the sensor update rate independently of the reporting interval.

Quantity	Calculation	Interpretation
FPGA period	1 / 100 MHz = 10 ns	One cycle of the board clock.
SPI clock	100 MHz / (2 × 50) = 1 MHz	50 clocks low and 50 high per bit.
XYZ burst	(1 + 1 + 6) × 8 = 64 bits	Command, address and six data bytes.
Wire time	64 bits / 1 MHz = 64 µs	Plus chip-select and controller overhead.
Ideal bus ceiling	1 MHz / 64 = 15,625 bursts/s	Assumes continuous clocks and zero gap.
Fresh sensor data	1 / 100 Hz = 10 ms	The chip updates the output registers.
PC report interval	10,000,000 / 100 MHz = 100 ms	The FPGA reads and reports ten times per second.

At 10 reports/s the host sequence has a 5 Hz Nyquist frequency. The 100 Hz sensor update rate alone does not prevent aliasing in that sequence. To retain higher frequencies, increase the reporting rate. To retain a 10 Hz stream, low-pass filter the higher-rate samples before decimation.

Signed reconstruction and scaling

Decode the register example from page 4 and compare it with the recorded terminal output.

Axis	Low, high	12-bit code	Signed calculation	Acceleration
X	58 00	0x058	88 counts	+0.088 g
Y	83 00	0x083	131 counts	+0.131 g
Z	67 FC	0xC67	3175 - 4096 = -921 counts	-0.921 g

For Z, join the low four bits of FC to the eight bits of 67. Bit 11 of C67 is one. In 12-bit two's complement the signed value is therefore the unsigned code minus 2^{12} (4096). The range of codes is -2048 to +2047.

```
X=+0088 Y=+0131 Z=-0921
```

The formatter sends a label, equals sign, sign and four decimal digits per axis. Two spaces and the final carriage return and line feed bring the total to 25 characters.

ASCII conversion and UART serialization

Each signed result is encoded as a sign and four decimal digits. The USB bridge delivers their bytes to the PC serial port.

Worked encoding of one character

The formatter separates decimal places by repeated subtraction and adds each digit to ASCII '0'. For the digit 8, 0x30 + 8 gives 0x38. This byte is binary 00111000 when written with its most significant bit on the left.

```
start      data bits in time order       stop

  0          0 0 0 1 1 1 0 0             1
```

UART sends the least significant data bit first. The transmitter keeps the line high between characters. With 868 FPGA clocks per bit, the rate is about 115207 baud, close to the PC setting of 115200.

At 8N1, each character uses 10 bits: one start, eight data and one stop. A complete line therefore occupies 25 × 10 / 115200 seconds, or about 2.17 ms.

Test environments and prerequisites

Extract the complete Lab10 directory before creating the project or launching the viewer.

Test	Equipment and software
Behavioral simulation	Vivado 2022.2, Artix-7 support and XSim. Runs without a board or Python.
Read UART text	Programmed Nexys4 DDR, USB data cable, USB serial driver and PuTTY (115200 8N1).
Replay the capture	Python with Tk, pySerial and Matplotlib, plus uart_capture.txt. Runs without a board.
Plot live XYZ	Same Python environment and a programmed board. Close PuTTY to release the port.

Python setup for the graphical viewer (Windows)

The reference PC uses 64-bit Python 3.9.2 with Tk 8.6, pySerial 3.5 and Matplotlib 3.9.0. For a fresh Python 3.11 installation, use the virtual-environment commands in the README. From the Lab10 directory, install the two libraries with the interpreter that will run the viewer.

```
python -m pip install -r requirements.txt
```

Behavioral verification in Vivado

Target the supplied Verilog design and use the SPI slave model and independent UART decoder.

```
vivado -mode batch -source create_project.tcl
```

1. Run the command from the directory containing the script. Open build/lab10.xpr. Under Design Sources, verify that lab10_top is the top-level module. Under Simulation Sources, select tb_lab10.

2. Run Behavioral Simulation with tb_lab10 and select Run All. Expand the DUT. Add ACL_CSN, ACL_SCLK, ACL_MOSI, ACL_MISO, x, y, z, report and UART_RXD_OUT to the waveform. Restart and run again to capture the signals from reset.

3. Find the three 24-clock writes and the first 64-clock read. Decode MOSI during the second write. The first simulated report should contain -2048, +2047 and zero. The bench checks two complete lines and reports PASS. Its shortened delays accelerate simulation.

Lab record **Label the opcode, address and six data-byte intervals on one SPI waveform.**

FPGA implementation and acquisition

Build for xc7a100tcsg324-1. The onboard sensor is already wired to the FPGA.

CLK100MHZ	CPU_RESETN	ACL_CSN	ACL_SCLK	ACL_MOSI	ACL_MISO	UART_RXD_OUT
E3	C12	D15	F15	F14	E15	D4

1. Generate the bitstream. Connect the powered board via PROG/UART. In Hardware Manager choose Open Target > Auto Connect and program the XC7A100T. Streaming starts automatically. CPU RESET restarts initialization if needed.

2. Open the board's serial port at 115200 baud, 8N1, flow control off. The tested PC used COM8. Observe ten lines, turn the board over, let it settle and capture ten more. Identify the axis with the largest sign change.

3. For a live plot, close the terminal to release the port. Run Start_XYZ.cmd, select the port and click Connect live. Read the graph in g. Replay capture displays the saved one-position experiment.

Experimental evidence and uncertainty

Separate protocol correctness, host arrival timing and transducer accuracy in the laboratory report.

Recorded series	X (counts)	Y (counts)	Z (counts)
Mean of 30 records	+89.33	+132.17	-929.13
Sample standard deviation	2.02	2.15	3.78

The one-position capture spans 2890 ms across 29 intervals, giving a mean host arrival interval of 99.66 ms. USB buffering and host scheduling affect the timestamps. For cycle-accurate period verification, measure successive burst starts in the RTL simulation.

Two-position estimate of scale and offset

```
b = (n_plus + n_minus)/2        S = (n_plus - n_minus)/(2 g)
```

Acquire stationary means with the same axis aligned successively at +1 g and -1 g. Report the offset in counts and sensitivity in counts/g. Misalignment reduces the estimated sensitivity.

Analysis and laboratory report

Submit one annotated SPI waveform, two terminal captures and a short interpretation of the results.

1. Annotate the 0x0B burst. Identify the rising edge at which the Z sign bit is captured. Show how an off-by-one edge or a swapped byte pair changes the reconstructed result.

2. Estimate offset and sensitivity for the reversed axis. Report the sample size, orientation and standard deviation. Discuss whether the measurements distinguish scale error from misalignment.

3. To report at 20 Hz, determine a new PERIOD. Check whether the first read still allows 40 ms to settle and calculate the UART transmission time as a percentage of the new interval.

Technical references

Analog Devices | ADXL362 Rev. G: Tables 1 and 9-13, SPI Commands and Register Details

Digilent | Nexys4 DDR Rev. C master XDC: clock, reset, accelerometer and UART pins

Logic Design Using HDL | Lab 04 addressed memory and Lab 08 UART exercises

Course reference: Dr. István Oniga, Reconfigurable Circuits, sensor-interfacing laboratory handout.